KATHMANDU DON BOSCO COLLEGE

Affiliated to Purbanchal University

Lazimpat, Kathmandu

HOSPITAL MANAGEMENT SYSTEM

USING C++ PROGRAMMING

(A Partial Fulfillment of II Semester Requirement for BIT)

Submitted To:

Department of Information Technology
Kathmandu Don Bosco College

Submitted By:

Group F

Sarthak Kuikel

Nabin Panday

Sushant Basnet

II Semester, BIT

October, 2026

RECOMMENDATION

This is to certify that Mr.Sarthak Kuikel, Mr.Nabin Panday, and Mr.Sushant Basnet have prepared this project entitled “Hospital Management System” for the partial fulfillment of the Bachelor of Information Technology (BIT) Degree II Semester under the supervision of the project supervisor.

I recommend this project for approval and acceptance.

Date: _____

.....

Abhishek Kafle

Project Supervisor / Faculty Member
Kathmandu Don Bosco College
Lazimpat, Kathmandu

APPROVAL SHEET

This is to certify that the project report entitled “Hospital Management System” submitted in partial fulfillment of the requirements of the Bachelor of Information Technology (BIT) program has been satisfactorily completed and is submitted for evaluation.

Submitted By:

i. Sarthak Kuikel

ii. Nabin panday

iii. Sushant Basnet

External Examiner: _____

Supervisor: _____

Exam Head: _____

Date: _____

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to the administration of Kathmandu Don Bosco College for providing us with the opportunity to undertake this second-semester project. The project has provided us with practical experience in software design, C++ programming, object-oriented design, web-based graphical user interface development, and file-based data storage.

We are deeply grateful to our project supervisor, Abhishek Kafle for his/her guidance, encouragement, and constructive suggestions throughout the development of the Hospital Management System. We also extend our appreciation to the faculty members of the Information Technology Department for their academic support.

Finally, we would like to thank our classmates, friends, and family members for their encouragement and support during the planning, implementation, testing, and documentation of the project.

Group F
BIT II Semester, 2026

ABSTRACT

This report presents the design and implementation of the Hospital Management System, a desktop-based application developed using C++ with a browser-based graphical interface and text-file data storage. The system is designed to demonstrate fundamental concepts of object-oriented programming, user authentication, role-based access control, patient record management, appointment scheduling, medical history recording, and automatic billing within a local application environment. The application provides three user roles: Admin, Receptionist, and Doctor. After login, each role sees a dashboard and a menu that contains only the functions it is allowed to use. The receptionist registers patients and books appointments; the doctor records medical visits with a diagnosis and medical notes and suggests tests or procedures; the admin manages doctor and receptionist accounts and views reports; and billing is generated automatically from the doctor's consultation fee and the fixed prices of the tests and procedures used. Patient, appointment, visit, service, bill, and user records are stored in plain text files and loaded into memory when the program starts. The C++ code is organized into model classes (User with Admin, Receptionist, and Doctor; Patient; Appointment; MedicalVisit; Service with Test and Procedure; and Bill), a HospitalSystem class that holds the data and business rules, and a small web server that exposes the operations to the interface. The interface is written in HTML, CSS, and JavaScript and is served by the program itself, so no database server or internet connection is required. Validation rules prevent double booking of a doctor, invalid dates and times, changes to paid bills, and actions that a role is not permitted to perform. The system was verified with 23 unit and system test cases, all of which passed.

The project is intended as an academic prototype rather than a production hospital information system. Important future improvements include secure password hashing, a database such as SQLite, network access for several computers, prescriptions and pharmacy management, bed management, an editable price list, audit logs, and PDF export of bills and reports.

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
BIT	Bachelor of Information Technology
C++	General-purpose programming language used for implementation
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
ER	Entity-Relationship
GUI	Graphical User Interface
HMS	Hospital Management System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
OOP	Object-Oriented Programming
PU	Purbanchal University
SDLC	Software Development Life Cycle
UI	User Interface

LIST OF FIGURES

Figure 3.1 Context Diagram

Figure 3.2 Data Flow Diagram

Figure 3.3 Entity-Relationship Diagram

Figure 3.4 Use Case Diagram

Figure 3.5 Sequence Diagram

Figure 3.6 Activity Diagram

Figure 3.7 System Flowchart

Figure 4.1 Login Interface

Figure 4.2 Admin Dashboard

Figure 4.3 Patients Page

Figure 4.4 Register Patient Window

Figure 4.5 New Appointment Window

Figure 4.6 Add Medical Visit Form

Figure 4.7 Patient History Timeline

Figure 4.8 Bill Window

Figure 4.9 Reports Page

Figure 5.1 Planned Project Schedule

LIST OF TABLES

Table 2.1 Functional Requirements

Table 2.2 Non-Functional Requirements

Table 2.3 Feasibility Analysis

Table 4.1 Hardware Requirements

Table 4.2 Software Requirements

Table 4.3 Unit Test Cases

Table 4.4 System Test Cases

Table 5.1 Objective-Based Evaluation

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Project Objectives 2
1.4 Project Scope 2
1.5 Project Limitations 3

CHAPTER 2: LITERATURE REVIEW **4**
2.1 Background 4
2.2 Research Based on Similar Project Concepts 4
2.3 Functional and Non-Functional Requirements 5
2.4 Feasibility Study 6

CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY **7**
3.1 Software Development Life Cycle 7
3.2 Context Diagram 7
3.3 Data Flow Diagram 8
3.4 Entity-Relation Diagram 9
3.5 Use Case Diagram 10
3.6 Sequence Diagram 11
3.7 Activity Diagram 12
3.8 System Flowchart 13

CHAPTER 4: IMPLEMENTATION **14**
4.1 Software and Hardware Requirements 14
4.2 System Implementation 14
4.3 Testing 17
4.4 Source Code 18
4.5 Outputs 18

CHAPTER 5: ANALYSIS AND EVALUATION **24**
5.1 Analysis of Output Obtained 24
5.2 Schedule Analysis: Gantt Chart 24
5.3 Strengths and Technical Observations 25

CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATION **26**
6.1 Problems Faced and Their Implementation with Limitation of the Project 26
6.2 Conclusion 26
6.3 Future Recommendation 26
6.4 Final Recommendation 27

REFERENCES **28**

APPENDICES **28**
Appendix A: User Manual (Summary) 83

CHAPTER 1: INTRODUCTION

1.1 Background

Hospitals and clinics handle many connected activities every day: registering patients, scheduling appointments with doctors, recording diagnoses and treatment, ordering tests, and preparing bills. In many small institutions these activities are still recorded in paper registers, which makes searching, correcting, and summarizing the information slow and error-prone. The Hospital Management System was developed to provide a small, self-contained environment in which patient registration, appointment booking, medical history recording, test and procedure ordering, and automatic billing can be demonstrated on a single computer without relying on a remote server or a database product.

The project is implemented in C++17. The core of the system is a set of classes that represent the real objects of a hospital: users with different roles, patients, appointments, medical visits, services such as tests and procedures, and bills. The program starts a small local web server using the single-header cpp-httplib library and opens the user interface in the web browser. The interface is written in HTML, CSS, and JavaScript and is stored inside the program, so the application is started by running a single executable file.

All records are stored in plain text files in a data folder next to the program. When the program starts for the first time it creates default accounts for an administrator, a receptionist, and two doctors, together with a predefined price list of tests and procedures. This design makes it possible to log in and test the complete workflow immediately after the application is launched.

1.2 Problem Statement

Paper-based or loosely organized hospital records make it difficult to find a patient's history, to avoid two appointments at the same time for one doctor, and to prepare correct bills. Staff members also see more information and functions than their job requires. The project addresses these problems by creating a local application that stores records in files and provides controlled workflows for each staff role.

i. Need for a single digital record for every patient instead of paper registers.

ii. Need for role-based access so that each staff member sees only the functions required by the job.

iii. Need to prevent scheduling conflicts such as two appointments for the same doctor at the same time.

iv. Need for a complete medical history of each patient, including visits, diagnosis, and tests.

v. Need for automatic billing from fixed prices to avoid manual calculation mistakes.

vi. Need for simple reports of patients, doctors, appointments, and billing for management.

1.3 Project Objectives

1.3.1 General Objective

To design and implement a desktop-based Hospital Management System using C++ and a browser-based interface that manages patients, appointments, medical visits, and billing for three user roles: Admin, Receptionist, and Doctor.

1.3.2 Specific Objectives

i. To design the system using object-oriented classes with inheritance and polymorphism.

ii. To provide login and a role-specific dashboard for the Admin, Receptionist, and Doctor.

iii. To register, search, edit, and delete patient records with input validation.

iv. To book and cancel appointments and to prevent double booking of a doctor.

v. To record medical visits with a diagnosis and notes and to display each patient's history as a timeline.

vi. To let doctors suggest tests and procedures from a fixed price list.

vii. To generate bills automatically, allow other charges, and update the payment status.

viii. To produce patient, doctor, appointment, and billing reports.

ix. To store all data persistently in text files.

x. To provide logout and clear error and status messages for user interaction.

1.4 Project Scope

The project scope is limited to the functionality implemented in the source code. It includes local data storage, login with three roles, patient management, appointment booking and cancellation, medical visits and patient history, test and procedure suggestion, automatic

billing, reports, staff account management, and logout. The system operates as a standalone application on one computer and does not connect to an insurance provider, payment gateway, pharmacy, laboratory system, cloud server, or any external network.

1.5 Project Limitations

i. The system is a local academic prototype and must not be used for real patient data.

ii. Passwords are stored as ordinary text; secure hashing is not implemented.

iii. Data is stored in text files and the design assumes one running copy of the program; simultaneous use from several computers is not supported.

iv. The built-in server listens only on the local computer (127.0.0.1).

v. Prescriptions, pharmacy, ward and bed management, inventory, laboratory results, and insurance are not implemented.

vi. The price list of tests and procedures is fixed at first start-up and cannot be edited from the interface; discounts are not supported.

vii. There is no session time-out or audit log, and only basic input validation is performed.

CHAPTER 2: LITERATURE REVIEW

2.1 Background

A hospital information system generally requires mechanisms for identifying patients, scheduling appointments, keeping clinical records, calculating charges, and controlling which staff member may perform which action. In a production environment these functions are distributed across secure client applications, application servers, databases, and external services. For an academic prototype, the same responsibilities can be represented locally using C++ classes, a small server, and a browser-based interface.

A browser-based interface was chosen because HTML, CSS, and JavaScript provide forms, tables, dialogs, and styling without any additional graphical library. The C++ program acts as the back end: it holds the data, applies the rules, and answers requests from the interface. The interface and the back end communicate through a small set of HTTP routes that exchange form data and JSON.

2.2 Research Based on Similar Project Concepts

Academic hospital and clinic management projects commonly separate interface responsibilities from data-management responsibilities. This separation improves maintainability because the interface can request operations from a data layer instead of containing the rules itself. The supplied project follows this principle through the HospitalSystem class, which owns the data and rules, and the web interface, which only sends requests and displays results.

The project also applies role-based access control. Each user account has a role, the role decides the menu that is shown, and the server checks the role again on every request, so a function that is hidden from a role cannot be reached by sending the request directly. A further concept used is automatic billing: when a visit is saved, a bill is created, and every test or procedure added to the visit copies its name and price into the bill, so later changes to the price list do not alter old bills.

2.3 Functional and Non-Functional Requirements

2.3.1 Functional Requirements

Table 2.1: Functional Requirements

ID	Function	Description
FR1	Login	Authenticate a user with username and password and open the dashboard for the user's role.
FR2	Role-Based Menu	Show Admin, Receptionist, or Doctor pages according to the role.
FR3	Register Patient	Accept name, date of birth, gender, phone, blood group, address, and emergency contact and assign a patient ID.
FR4	Manage Patients	Search, view, edit, and (Admin only) delete patients; deletion also removes related records.
FR5	Book Appointment	Book an appointment for a patient with a doctor at a date and time; reject double booking.
FR6	Cancel Appointment	Cancel a scheduled appointment.
FR7	Add Medical Visit	Doctor records diagnosis and notes for a patient and may link a scheduled appointment.
FR8	Patient History	Display all visits of a patient as a timeline.
FR9	Suggest Test or Procedure	Doctor adds a test or procedure from the price list to a visit.
FR10	Billing	Create a bill for each visit, add service prices and other charges, mark the bill paid or unpaid, and print it.
FR11	Staff Management	Admin adds, deletes, and resets the password of doctors and receptionists.
FR12	Reports	Admin views patient, doctor, appointment, and billing reports.
FR13	Logout	End the session and return to the login screen.

2.3.2 Non-Functional Requirements

Table 2.2: Non-Functional Requirements

Requirement	Description
Usability	Simple form-based interface with a menu, tables, dialogs, and clear status messages.
Reliability	Invalid input and forbidden actions return an error message without changing data; data is saved after every change.
Performance	All operations are local, in-memory operations and are expected to respond immediately on a normal computer.
Maintainability	Code is divided into model classes, a HospitalSystem class, a server file, and separate interface files.
Portability	Written in standard C++17; the same source builds on Windows, Linux, and macOS.
Security	Role checks are enforced on the server; login tokens identify the session. Passwords are not hashed in this prototype.
Data Integrity	Special characters are removed before saving so that records cannot corrupt the file format; related records are removed together.

2.4 Feasibility Study

Table 2.3: Feasibility Analysis

Feasibility	Assessment	Conclusion
Technical	C++17, the cpp-httplib header, HTML, CSS, and JavaScript are used. The project is divided into model, logic, server, and interface parts.	Feasible for an academic desktop prototype.
Economic	All tools used are free; no database server, license, or internet service is required.	Very low cost.
Operational	Each role has a simple menu and forms that match its daily tasks.	Suitable for demonstration and learning.
Schedule	The work is divided into design, classes, server, interface, testing, and documentation over about twelve weeks.	Feasible within the semester.

CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY

3.1 Software Development Life Cycle

The Software Development Life Cycle provides a structured sequence for planning, designing, implementing, testing, and documenting software. For this project, a Waterfall-style academic workflow is suitable because the requirements are small and clearly identifiable: login, patients, appointments, medical visits, billing, reports, and logout.

3.1.1 Used Model: Waterfall Model

i. Requirements Analysis: Identify the required hospital functions and the three user roles.

ii. System Design: Define the classes, data files, interfaces, and data flows.

iii. Implementation: Develop the C++ classes, the server, and the web interface.

iv. Testing: Test validation rules, permissions, billing, and data persistence.

v. Evaluation: Compare the implemented functions against the project objectives.

vi. Documentation: Prepare the project report, user manual, and source-code appendix.

3.2 Context Diagram

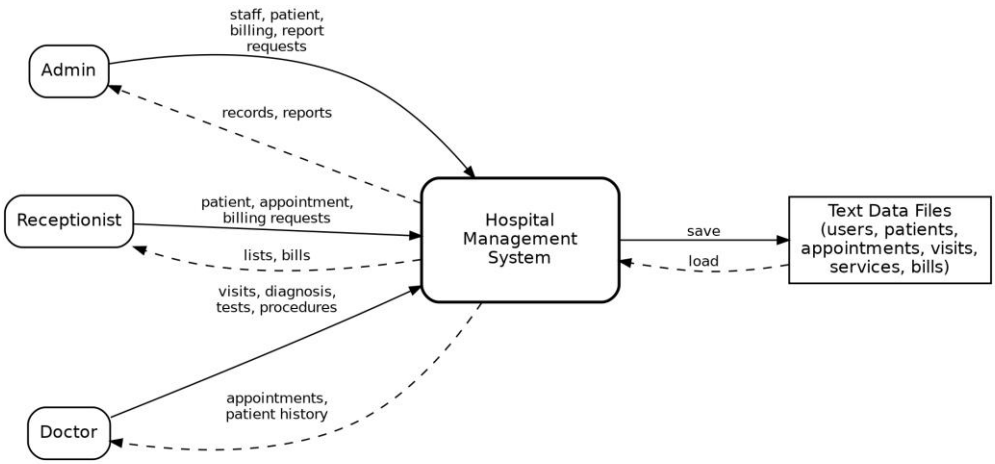


Figure 3.1: Context Diagram of Hospital Management System

The context diagram represents the Admin, Receptionist, and Doctor as the external actors and the Hospital Management System as the central application. Each actor sends requests appropriate to the role and receives records, lists, or reports in return. The system exchanges data only with the local text files; unlike an online hospital service, the project does not show an external payment gateway, insurance provider, or remote server.

3.3 Data Flow Diagram

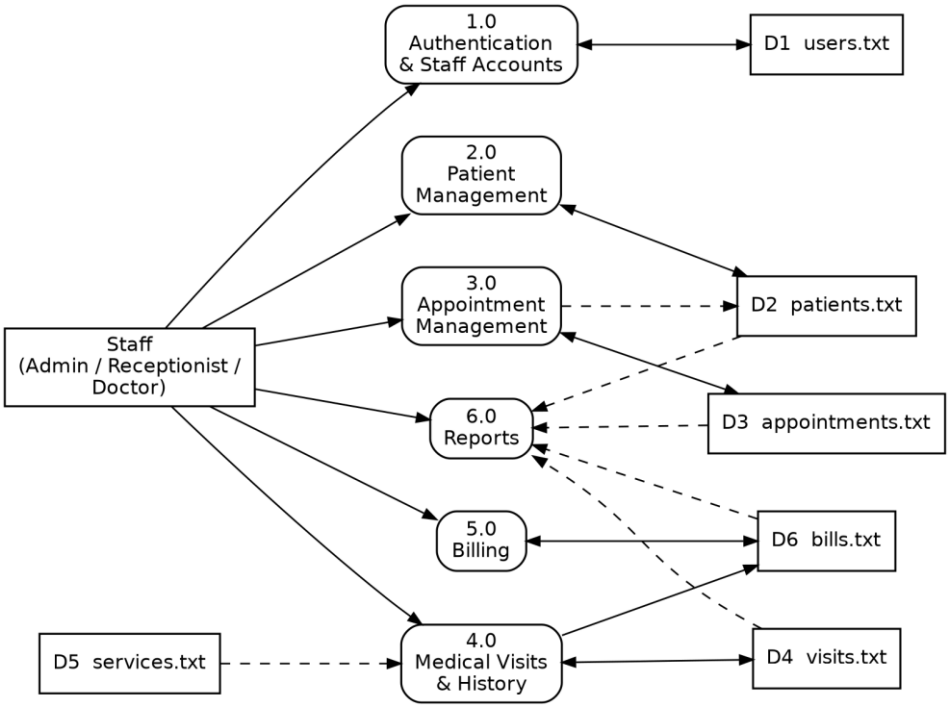


Figure 3.2: Level-1 Data Flow Diagram

The DFD divides the system into six processes: authentication and staff accounts, patient management, appointment management, medical visits and history, billing, and reports. Staff input enters through the interface. Each process reads and updates its data store: users, patients, appointments, visits, and bills. The visits process also reads the services store for prices and creates the bill record when a visit is saved, and the reports process reads from the stores without changing them.

3.4 Entity-Relation Diagram

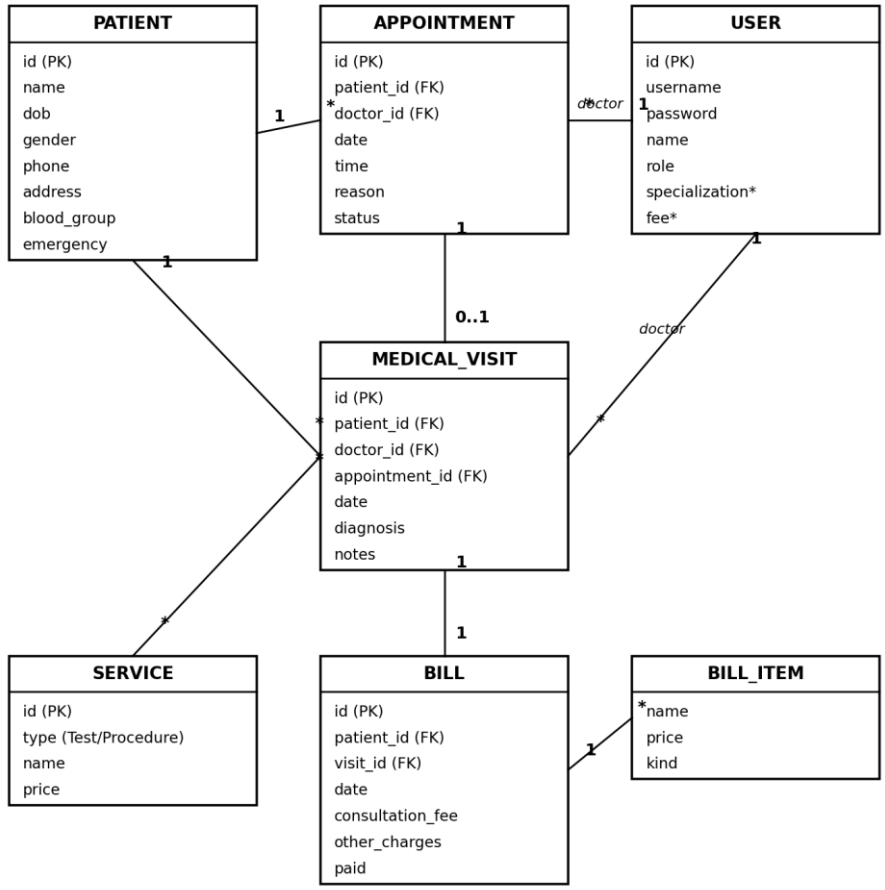


Figure 3.3: Entity-Relation Representation

The data model contains seven entities. A patient can have many appointments and many medical visits. A doctor (a user whose role is Doctor) can have many appointments and visits. A medical visit may be linked to one appointment, produces exactly one bill, and can include many services; a service can appear in many visits. A bill contains many bill items, and each item stores the service name, price, and kind at the time of billing. In the implementation the services chosen for a visit are stored as a list of service IDs in the visit record.

3.5 Use Case Diagram

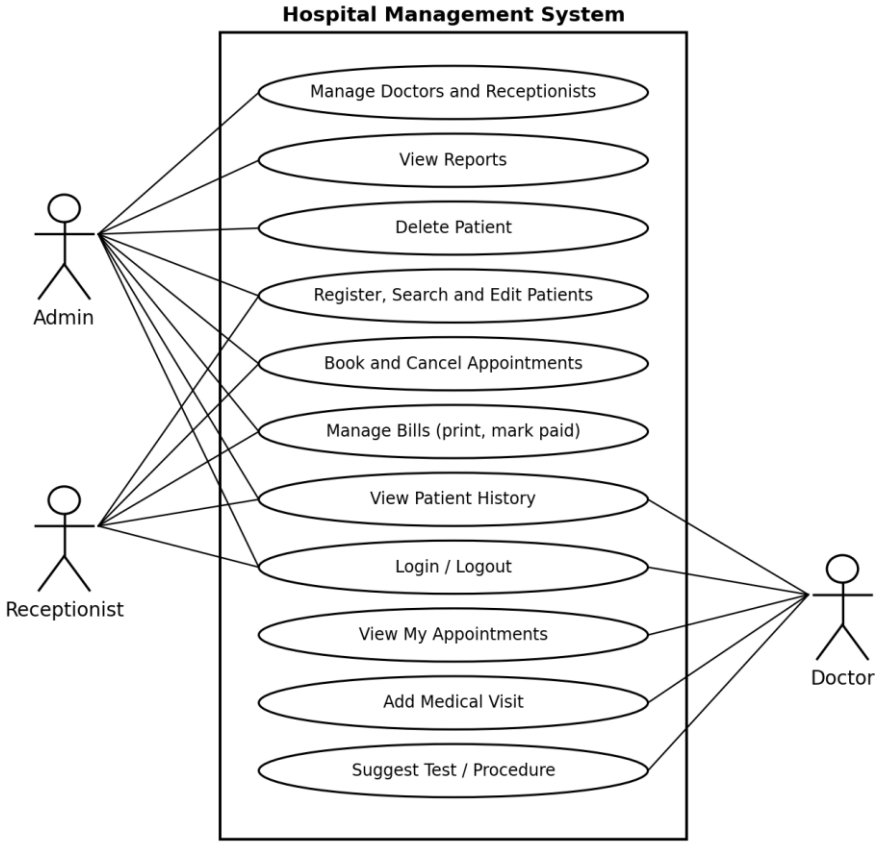


Figure 3.4: Use Case Diagram

All three actors can log in and log out and can view a patient's history. The Admin can additionally manage doctors and receptionists, delete patients, and view reports, and shares patient, appointment, and billing functions with the Receptionist. The Doctor views his or her own appointments, adds medical visits, and suggests tests and procedures.

3.6 Sequence Diagram

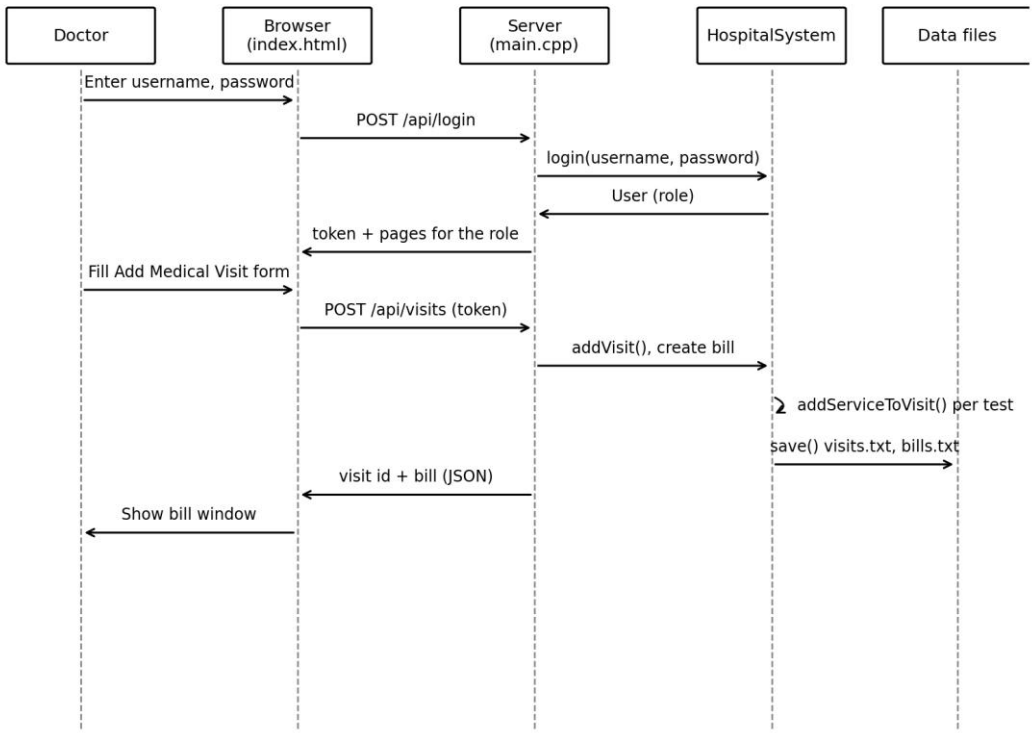


Figure 3.5: Sequence Diagram for Login and Medical Visit Billing

The sequence shows the interaction between the doctor, the browser interface, the server, the HospitalSystem class, and the data files. Login sends the username and password to the server, which asks HospitalSystem to authenticate and returns a token and the menu for the role. When the doctor saves a visit, the server calls addVisit() to create the visit and its bill, calls addServiceToVisit() for each selected test or procedure, saves the files, and returns the bill to the interface.

3.7 Activity Diagram

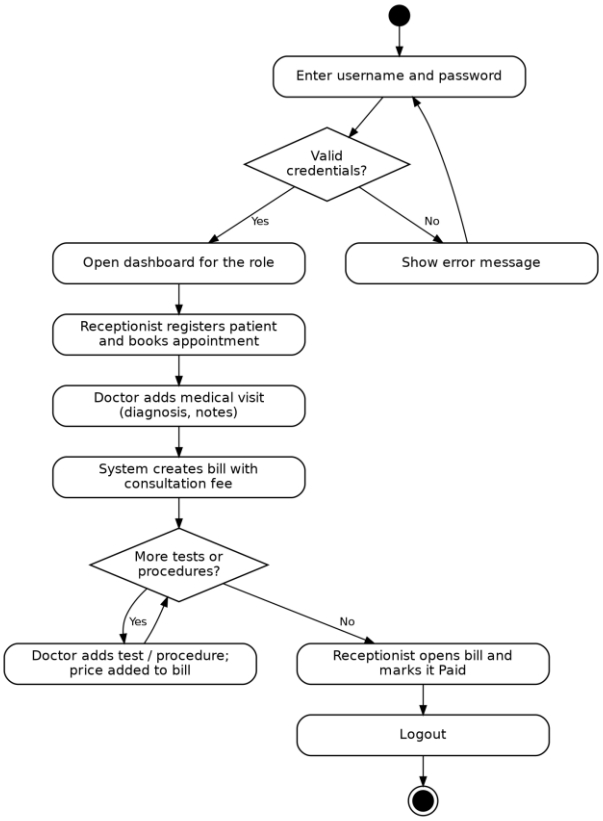


Figure 3.6: Activity Diagram

The activity diagram follows one patient through the system: credentials are validated, the receptionist registers the patient and books an appointment, the doctor adds a medical visit, the system creates the bill, the doctor may add more tests or procedures, the receptionist marks the bill as paid, and the user logs out.

3.8 System Flowchart

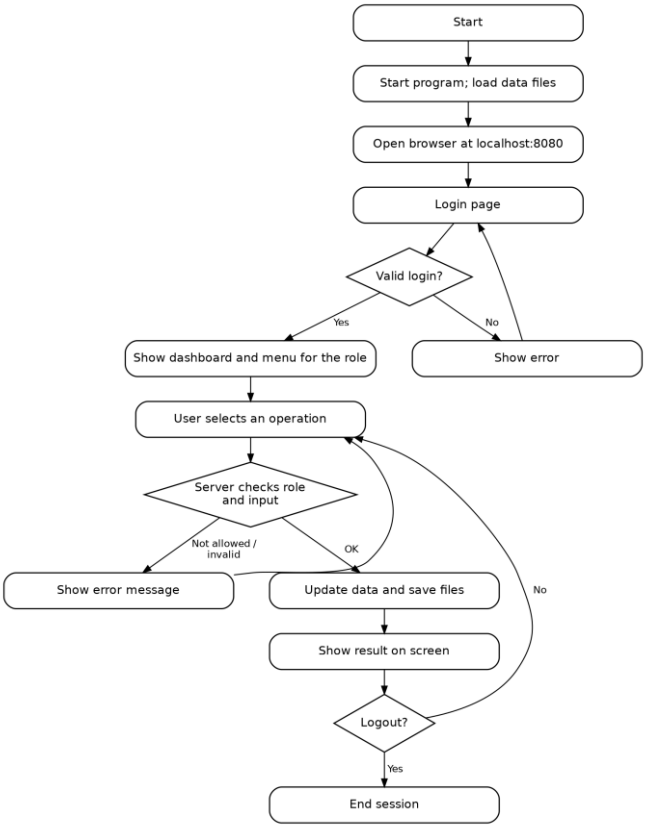


Figure 3.7: High-Level System Flowchart

The flow begins with starting the program and loading the data files. The browser opens the login page. After successful login, the dashboard and menu for the role are shown. For every operation the server checks the role and the input; invalid or forbidden requests produce a message, and valid requests update the data, save the files, and show the result. The session ends when the user logs out.

CHAPTER 4: IMPLEMENTATION

4.1 Software and Hardware Requirements

4.1.1 Hardware Requirements

Table 4.1: Hardware Requirements

Component	Recommended Minimum
Processor	Dual-core processor or equivalent
RAM	4 GB or more
Storage	At least 100 MB free space for the program, data, and build files
Display	1366 x 768 resolution or higher
Input devices	Keyboard and mouse

4.1.2 Software Requirements

Table 4.2: Software Requirements

Software/Tool	Purpose
C++17 (g++ / MinGW)	Programming language standard and compiler used for the back end
cpp-httplib v0.18.7	Single-header HTTP server library included as httplib.h
HTML, CSS, JavaScript	Graphical user interface that runs in the browser
Web browser	Displays the interface (any modern browser)
Text files	Persistent storage of users, patients, appointments, visits, services, and bills
Python 3 (optional)	Only needed to regenerate the embedded page after editing interface files
Operating system	Windows 10/11 (the source also builds on Linux and macOS)

4.1.3 Language Used

The back end is written in C++17 and compiled with g++ using the command g++ -std=c++17 -O2 -o hms.exe main.cpp -lws2_32 on Windows. The interface is written in HTML, CSS, and JavaScript. The files in the web folder are combined into Page.h by the make_page.py script so that the interface is embedded in the executable, and the program serves it at the address http://localhost:8080.

4.2 System Implementation

The implementation is divided into four responsibilities. The model classes in the models folder represent the hospital objects. The HospitalSystem class in services/HospitalSystem.h owns all data, loads and saves the files, and applies the business rules. The main.cpp file starts the server and defines the routes that connect the interface to HospitalSystem. The web folder contains the interface, which is divided into one JavaScript file for each page.

4.2.1 Data Loading and Initialization

The HospitalSystem constructor creates the data folder with std::filesystem and calls load(). The load() function reads users.txt, patients.txt, appointments.txt, visits.txt, bills.txt, and services.txt, and converts every line into an object using the fromRecord() functions of the classes. Each line is one record and fields are separated by the character |. If no users exist, four default accounts are created; if no services exist, six predefined tests and procedures are created. The save() function writes every list back to its file and is called after each change, so the data on disk is always current.

4.2.2 Login and Role-Based Access

The login route calls HospitalSystem::login(), which compares the username and password with the stored users. On success the server creates a random token, stores it with the user ID, and returns the token together with the menu of the role. The menu comes from the virtual function pages() of the User class, which is overridden by Admin, Receptionist, and Doctor, so the role decides the menu through polymorphism. Every other route is wrapped by the route() function in main.cpp, which checks the token and the allowed roles before running the request and answers 401 when the user is not logged in and 403 when the role is not allowed.

4.2.3 Patient Registration and Management

The patient routes read the form fields and validate them: all fields must be present and the date of birth must be in the format YYYY-MM-DD. HospitalSystem::addPatient() assigns the next ID (P001, P002, and so on) and saves the record. Text entered by users passes through clean(), which replaces the characters | ~ ; and line breaks with spaces so that they cannot break the file format. Editing updates the stored object. Deleting a patient calls deletePatient(), which also removes the patient's appointments, visits, and bills so that no orphan records remain. Only the Admin may delete patients.

4.2.4 Appointment Booking

The appointment route checks that the patient and the doctor exist, that the date and time are valid, and that a reason is entered. The slotTaken() function then checks whether the same doctor already has a scheduled appointment at that date and time and rejects the booking if so. A new appointment has the status Scheduled, and a scheduled appointment

can be cancelled. When a doctor saves a visit linked to an appointment, the status becomes Completed.

4.2.5 Medical Visits and Billing

When a doctor saves a visit, the route validates the patient, the diagnosis, the notes, and the optional appointment, and then calls HospitalSystem::addVisit(). This function assigns the visit ID, finds the doctor, and creates a bill that starts with the doctor's consultation fee. For every selected test or procedure the route calls addServiceToVisit(), which looks up the fixed price in the service list, adds the service ID to the visit, and adds a bill item containing the name, price, and kind. The Bill class calculates the total as the consultation fee plus the test charges, the procedure charges, and any other charges. A bill that is marked as paid is locked: further services and charges are rejected. A doctor can add services only to his or her own visits.

4.2.6 Reports

The reports route is available only to the Admin. It collects the number of visits for every patient, the number of appointments and visits for every doctor, the counts of scheduled, completed, and cancelled appointments, and the paid, unpaid, and total amounts of all bills. The interface shows these as four separate reports: patient, doctor, appointment, and billing. Money is shown only in the billing report.

4.2.7 Web Interface

The interface consists of index.html (page structure), style.css (styling), and JavaScript files in the js folder. The file core.js contains shared helper functions, the api() function that sends requests with the login token, the login and menu code, and the start-up code. Each page has its own file: dashboard.js, patients.js, appointments.js, history.js, visit.js, bills.js, reports.js, and staff.js. A page function requests data with api(), builds a table or form, and displays it in the content area. Forms open in dialog windows, and the Add Medical Visit form shows a live estimate of the bill total as tests and procedures are ticked.

4.2.8 Logout

The logout route removes the token from the server, and the interface clears the saved token, hides the application screen, and shows the login screen again. After logout, any request using the old token is answered with 401.

4.3 Testing

4.3.1 Unit Testing

Table 4.3: Unit Test Cases

ID	Test	Expected Result	Status
UT01	Login with a wrong password	Login refused; “Invalid username or password.” is shown	Pass
UT02	Register a patient with an empty field	Patient not created; “Please fill in all fields.” is shown	Pass
UT03	Register a patient with an invalid date of birth	Rejected; date-format message is shown	Pass
UT04	Register two valid patients	Patients created with IDs P001 and P002	Pass
UT05	Appointment with an invalid time (25:99)	Rejected; “Time must be HH:MM.” is shown	Pass
UT06	Book the same doctor at the same date and time twice	Second booking rejected	Pass
UT07	Appointment for an unknown patient	Rejected; a patient must be selected	Pass
UT08	Save a visit with Blood Test and X-Ray (consultation Rs. 500)	Bill total is Rs. 1,600	Pass
UT09	Add a service to a paid bill	Rejected; bill is already paid	Pass
UT10	Add staff with an existing username	Rejected; “Username already exists.”	Pass
UT11	Add other charges of amount 0	Rejected; a valid amount is required	Pass
UT12	Add Rs. 200 other charges to an unpaid bill of Rs. 650	Bill total becomes Rs. 850	Pass

4.3.2 System Testing

Table 4.4: System Test Cases

ID	Scenario	Expected Behavior	Result
ST01	Register patient, book appointment, save visit, create bill, mark paid	Complete workflow succeeds; bill total Rs. 1,600	Pass
ST02	Doctor opens the bills list	Refused (HTTP 403)	Pass
ST03	Receptionist deletes a patient	Refused (HTTP 403)	Pass
ST04	Request sent without a login token	Refused (HTTP 401)	Pass
ST05	Save a visit linked to an appointment	Appointment status becomes Completed	Pass
ST06	Doctor adds a service to another doctor's visit	Refused; visit not found	Pass
ST07	Doctor views appointments	Only the doctor's own appointments are listed	Pass
ST08	Receptionist opens reports	Refused (HTTP 403)	Pass
ST09	Admin deletes a doctor who has records	Refused; records are protected	Pass
ST10	Close and restart the program	Patients and bills are still present	Pass
ST11	Admin deletes a patient	Patient's appointments, visits, and bills are also removed	Pass

All 23 test cases were executed against the running program by sending requests to its server and checking the replies, and the results above are the observed results. In addition, the interface was operated in a web browser through an automated session that covered

login, patient registration, appointment booking, medical visit, bill, reports, and staff pages for the Admin, Receptionist, and Doctor roles, and no errors were reported.

4.4 Source Code

All C++ files are written in a simple style and documented in plain language. Each file starts with a banner that explains its purpose, the complex logic is explained in numbered steps inside the comments, and every feature is marked with a tag from FEATURE 1 (login) to FEATURE 11 (role check) so that it can be found quickly; Appendix B lists where each feature is located. The complete source code is included in the appendices of this report and is also submitted separately with the project. The principal files are main.cpp, HospitalSystem.h, the model headers (User.h, Patient.h, Appointment.h, MedicalVisit.h, Test.h, Procedure.h, Bill.h, and Util.h), index.html, style.css, and the JavaScript files core.js, dashboard.js, patients.js, appointments.js, history.js, visit.js, bills.js, reports.js, and staff.js.

4.5 Outputs

The following figures are screenshots of the running application. They were captured with sample data and show the main screens of the three roles.

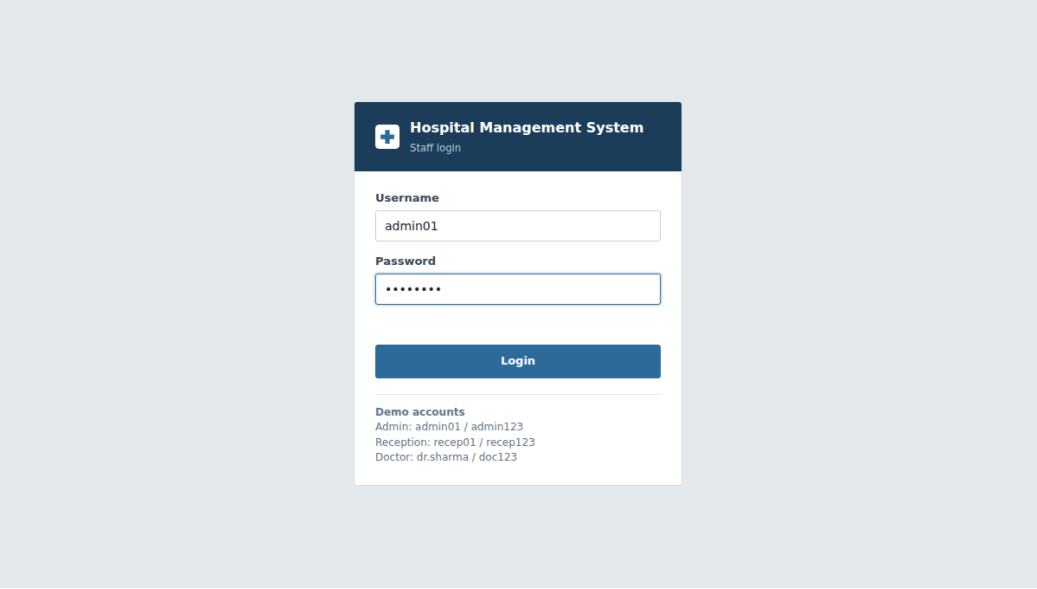


Figure 4.1: Login Interface

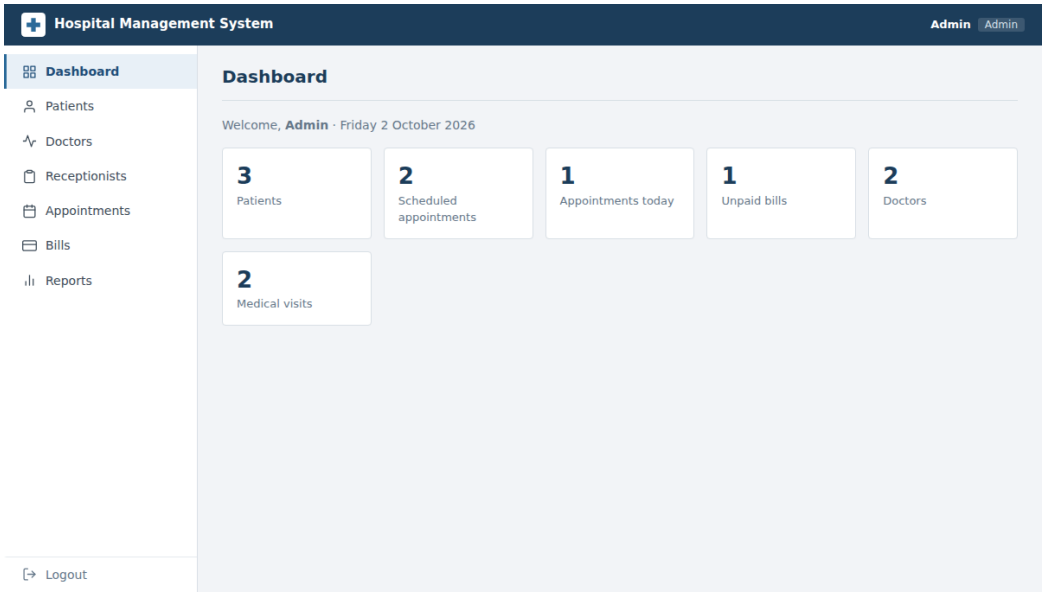


Figure 4.2: Admin Dashboard

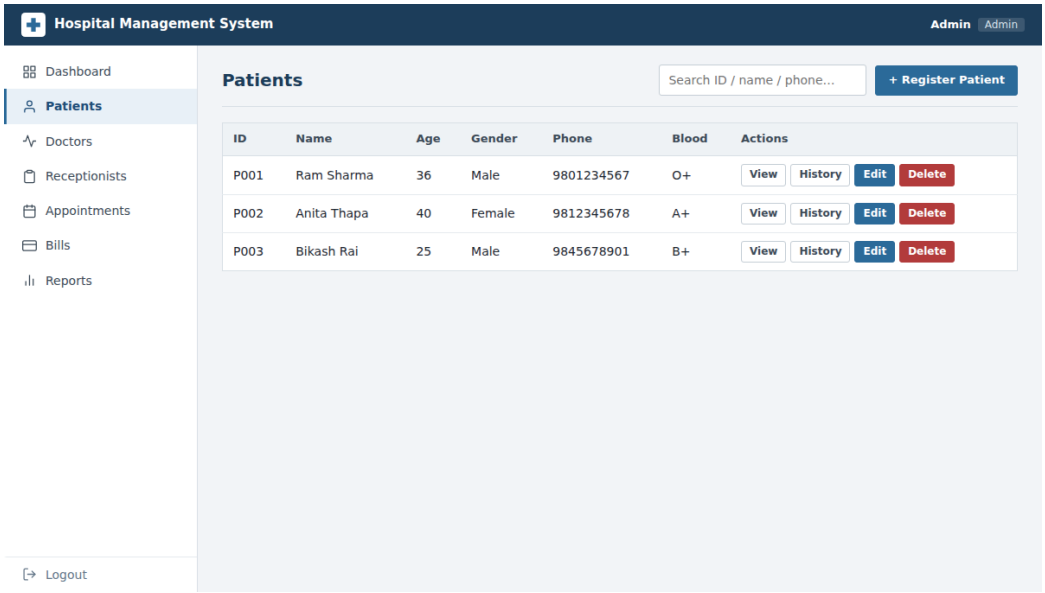


Figure 4.3: Patients Page

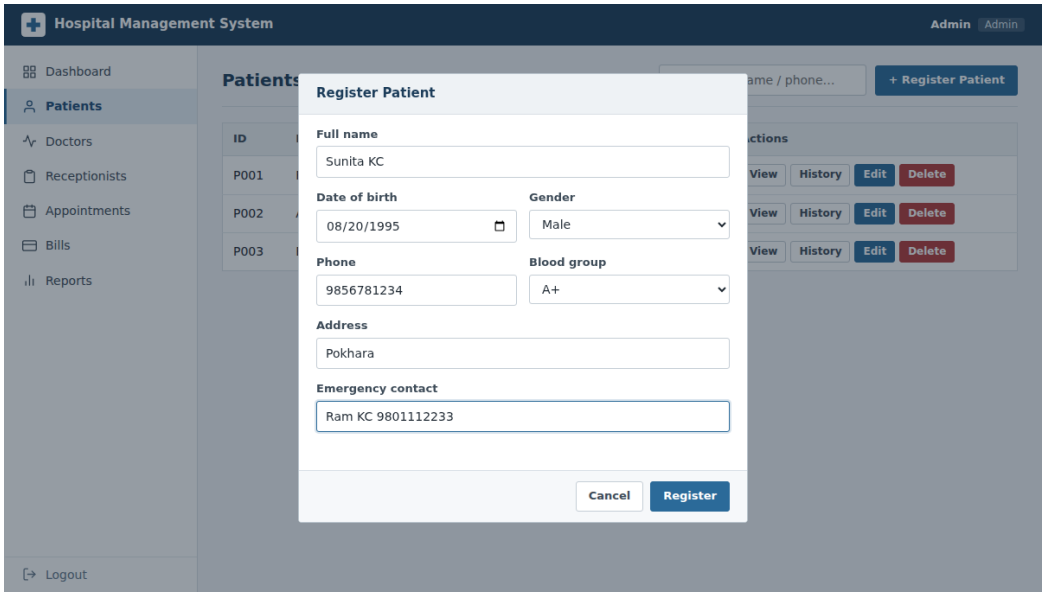


Figure 4.4: Register Patient Window

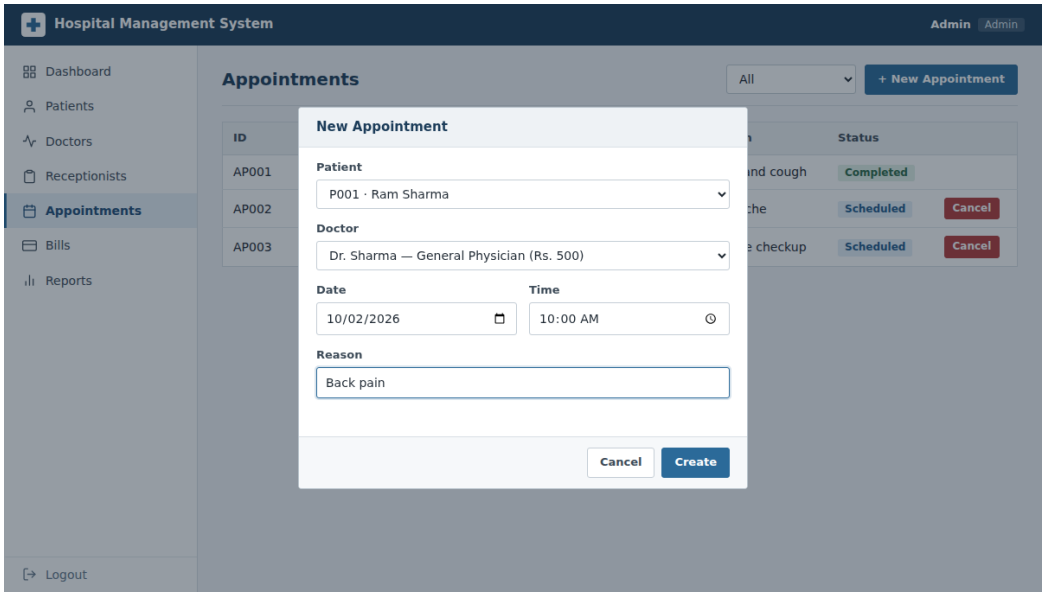


Figure 4.5: New Appointment Window

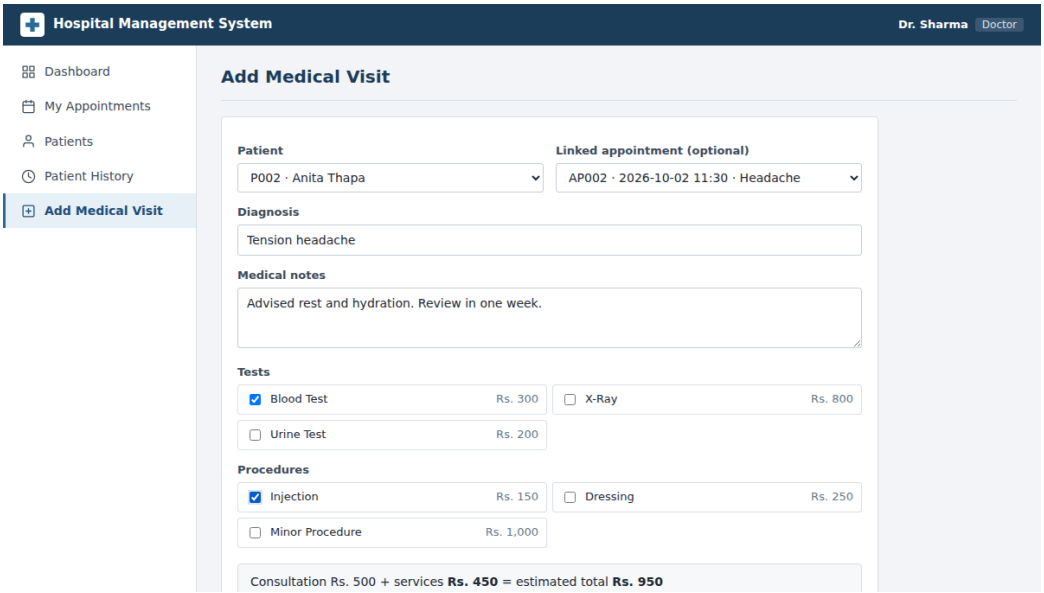


Figure 4.6: Add Medical Visit Form (Doctor)

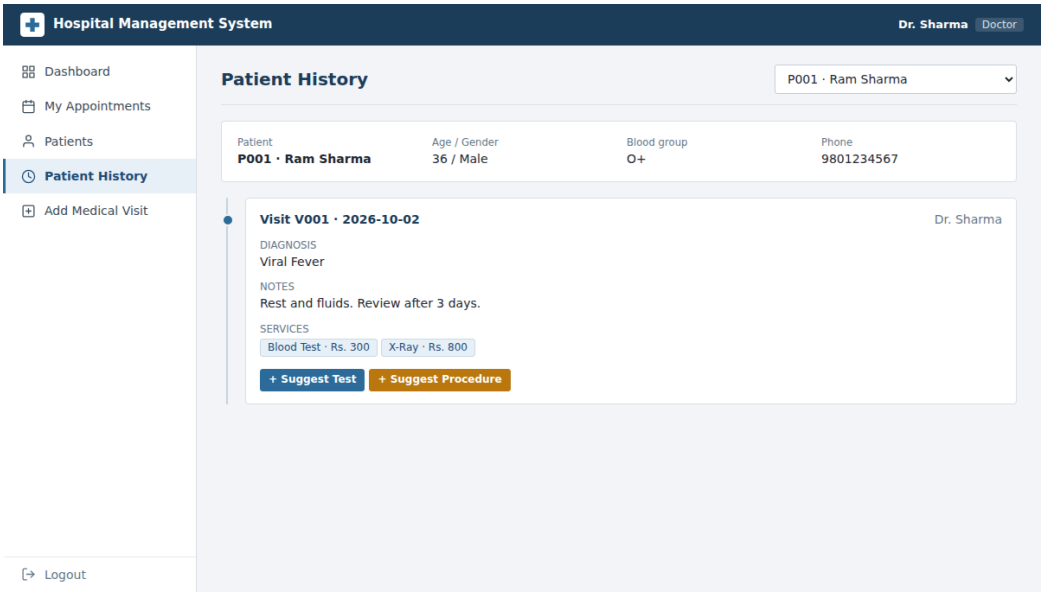


Figure 4.7: Patient History Timeline (Doctor)

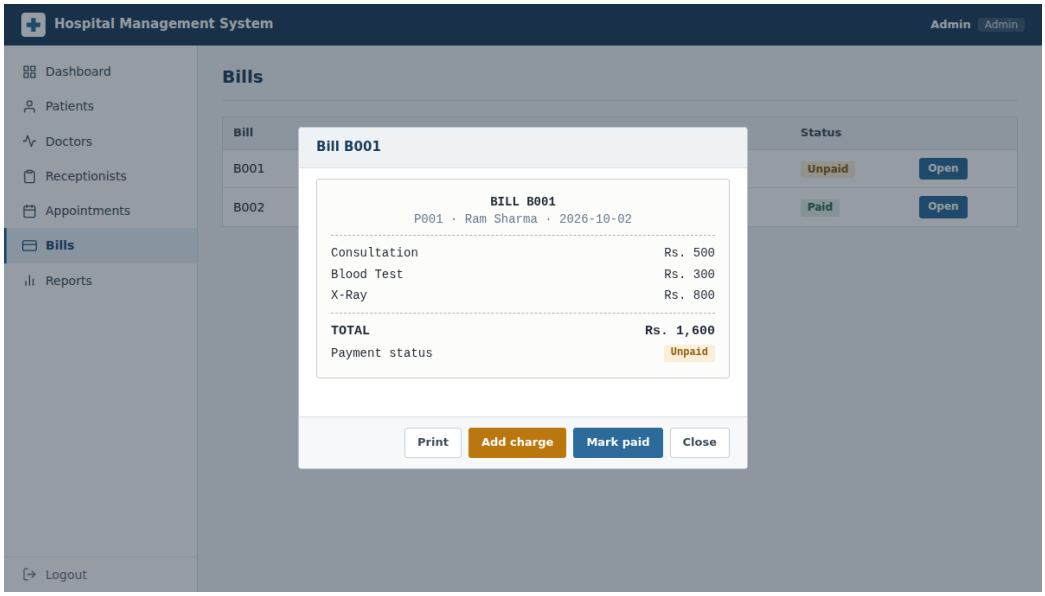


Figure 4.8: Bill Window

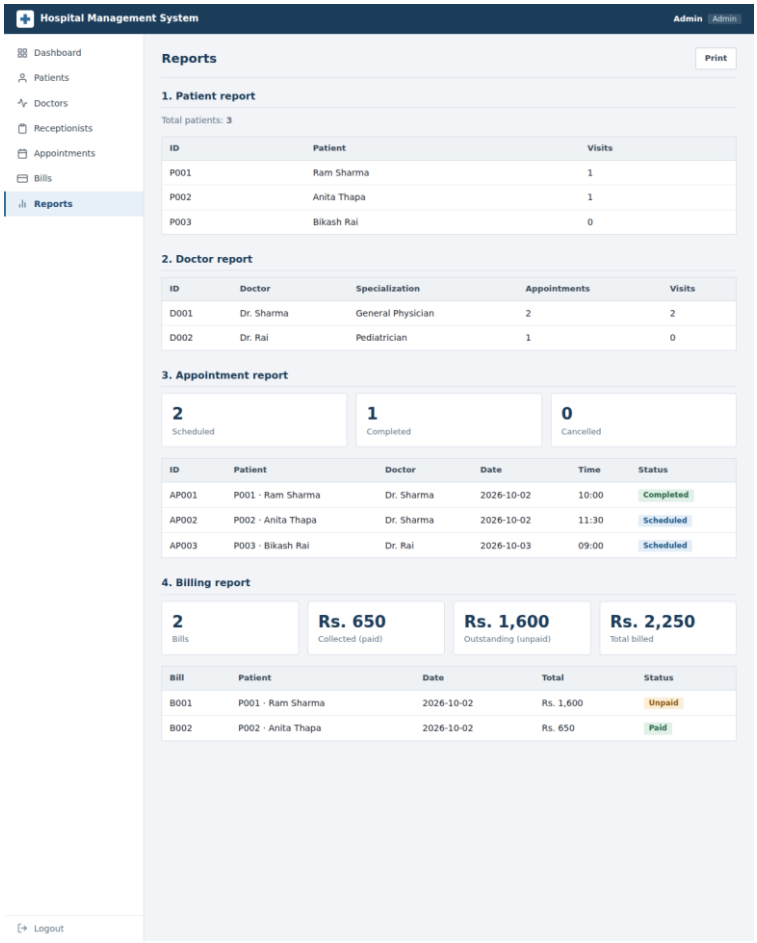


Figure 4.9: Reports Page (Admin)

CHAPTER 5: ANALYSIS AND EVALUATION

5.1 Analysis of Output Obtained

The implementation can be evaluated against the predefined objectives by tracing each objective to the corresponding functionality in the source code and to the test results in Chapter 4. The model classes represent the hospital objects, HospitalSystem applies the rules, the routes in main.cpp enforce the roles, and the interface displays the results.

Table 5.1: Objective-Based Evaluation

Objective/Feature	Analysis	Status
Login and roles	HospitalSystem::login(), User::pages(), and the route() role check are implemented.	Implemented
Patient management	Register, search, edit, and Admin-only delete with cascade removal are implemented.	Implemented
Appointment booking	Booking, cancelling, and double-booking prevention are implemented.	Implemented
Medical visits and history	addVisit() and the history timeline are implemented.	Implemented
Tests and procedures	Fixed price list and addServiceToVisit() are implemented.	Implemented
Automatic billing	A bill is created for each visit; totals, other charges, and paid status are implemented.	Implemented
Reports	Patient, doctor, appointment, and billing reports are implemented.	Implemented
Data persistence	Text files are loaded at start-up and saved after every change.	Implemented
Prescriptions and pharmacy	No prescription or pharmacy module is supplied.	Not implemented
Production-grade security	Password hashing, session time-out, and audit logging are not supplied.	Not implemented
Multi-computer access	The server listens only on the local computer.	Not implemented

5.2 Schedule Analysis: Gantt Chart

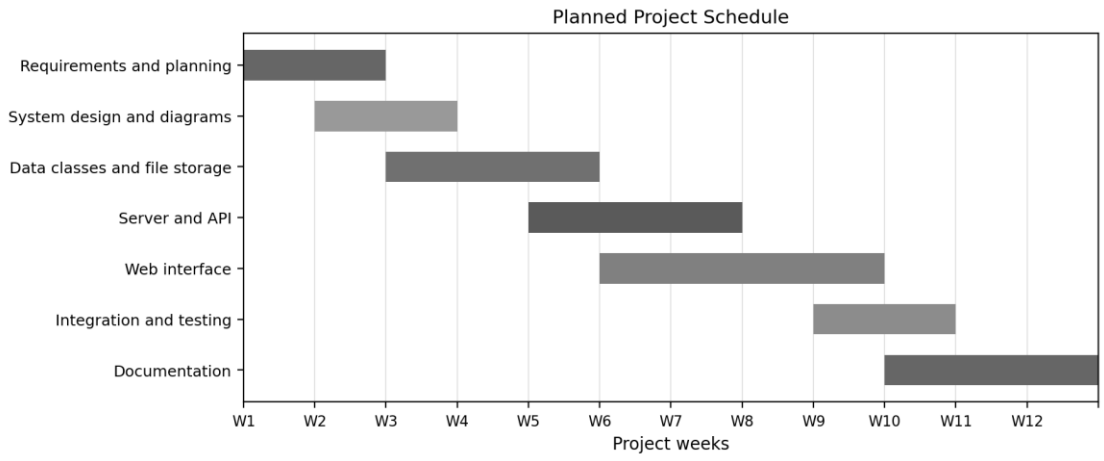


Figure 5.1: Planned Project Schedule

The schedule is presented as an academic project plan covering requirements, design, data classes and file storage, server and API, web interface, integration and testing, and documentation. It is a planning representation rather than a verified record of exact working hours.

5.3 Strengths and Technical Observations

i. The project uses object-oriented design with inheritance (User, Service) and polymorphism (pages() and type()).

ii. The interface is separated from the data and rules; the interface only sends requests and shows results.

iii. Role permissions are enforced on the server as well as in the menu.

iv. Billing is automatic, and each bill keeps the prices that applied at the time of billing.

v. Validation prevents invalid dates and times, double booking, empty fields, and changes to paid bills.

vi. Data is stored in readable text files that need no database software.

vii. The whole application runs from a single executable file with no installation.

viii. All 23 test cases passed in the final test run.

ix. The C++ code is documented with plain-language comments, numbered explanations of the main steps, and FEATURE tags, so each function can be found and explained easily.

CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATION

6.1 Problems Faced and Their Implementation with Limitation of the Project

A major challenge was keeping related records consistent. A bill must always belong to a visit, and a visit to a patient, so deleting a patient had to remove the related appointments, visits, and bills together. Another challenge was keeping the file format safe: a name or note containing the separator character could split a record into wrong fields, so user text is cleaned before saving. Controlling access was also important: hiding a menu item is not enough, so the server checks the role on every request. Finally, on some Windows computers the security policy blocked a downloaded, unsigned executable file; the solution was to compile the program locally with g++.

The current implementation remains an academic prototype. It does not provide a remote service, prescriptions, pharmacy, ward management, insurance, secure password hashing, session time-out, or an audit trail. Passwords are stored as text, which should not be used for a real hospital service.

6.2 Conclusion

The Hospital Management System demonstrates the integration of C++ object-oriented programming, a web-based graphical interface, and file-based data storage in a single application. The project successfully implements the main workflow of a small hospital: login with three roles, patient registration, appointment booking, medical visits with history, ordering of tests and procedures, automatic billing, reports, and logout.

The separation between the model classes, the HospitalSystem class, the server, and the interface makes the implementation easy to understand and maintain. The role checks, validation rules, and automatic billing provide a sound academic demonstration of application logic. The project therefore meets its primary educational purpose of applying programming and design concepts to a practical software problem.

6.3 Future Recommendation

i. Store passwords as secure hashes and add a session time-out.

ii. Replace the text files with a database such as SQLite.

iii. Allow other computers on the hospital network to connect to the server.

iv. Add prescriptions, pharmacy, and medicine inventory.

v. Add ward and bed management and doctor schedules or leave.

vi. Let the Admin edit the price list of tests and procedures from the interface.

vii. Add discounts, insurance, and partial payments to billing.

viii. Export bills and reports to PDF and send appointment reminders.

ix. Add an audit log that records who changed what and when.

x. Add automated regression tests and backup and restore of the data folder.

6.4 Final Recommendation

Before submission, the group should run the final build on the target computer, repeat the test cases, and have the recommendation and approval pages signed by the appropriate college personnel. The sample data used for the screenshots should be removed by deleting the data folder before the system is used for demonstration, so that the default accounts and price list are created again.

REFERENCES

[1] Y. Hirose, “cpp-httplib: A C++ header-only HTTP/HTTPS server and client library,” GitHub repository, accessed 2026.

[2] B. Stroustrup, The C++ Programming Language, 4th ed. Boston, MA, USA: Addison-Wesley, 2013.

[3] cppreference.com, “Filesystem library (C++17),” C++ Reference, accessed 2026.

[4] Mozilla, “Fetch API,” MDN Web Docs, accessed 2026.

[5] Kathmandu Don Bosco College, Department of Information Technology, “Project Report Format for BIT,” provided academic guideline.

APPENDICES

Appendix A: User Manual (Summary)

Login

1. Start hms.exe from the project folder; the browser opens at http://localhost:8080.

2. Enter the username and password. The role is decided by the account.

3. The dashboard and menu of the role are displayed.

Admin

The Admin can manage patients, doctors, receptionists, appointments, billing, and reports.

Receptionist

The Receptionist can register and search patients, edit patient details, manage appointments, view patient history, and manage billing.

Doctor

The Doctor can view own appointments and patient history, add medical visits, record diagnosis and medical notes, and suggest tests or procedures.

Billing

A bill is created for every visit from the consultation fee plus tests, procedures, and other charges. The system calculates the total and the Admin or Receptionist updates the payment status. A paid bill cannot be changed.

Logout

Select Logout at the bottom of the left menu to safely end the session and return to the login screen.

Default accounts

Role	Username	Password
Admin	admin01	admin123
Receptionist	recep01	recep123
Doctor	dr.sharma / dr.rai	doc123